Factorizer: A Scalable Context Modeling for Medical Image Segmentation

Pooya Ashtari *[a] and Sabine Van Huffel[a]

[a] *Department of Electrical Engineering (ESAT), KU Leuven*

1. Introduction

Centroid-based algorithms, such as k-means and FCM, are scalable, efficient

Contributions. first deep model with matrix factorization layers for medical image segmentation. different from HamNet, factorizer can exploit local context.

Notation. Lets briefly overview notation. We denote vectors by boldface lower-case letters, e.g., $\mathbf{x}$, matrices by boldface upper-case letters, e.g., $\mathbf{X}$, and tensors by boldface calligraphic letters, e.g., $\boldsymbol{\mathcal{X}}$. Elements in a matrix (tensor) are denoted by $\mathbf{X}[i, j]$ ($\boldsymbol{\mathcal{X}}[i_1, \ldots, i_N]$). The ith row and jth column of a matrix is denoted by $\mathbf{X}[i, :]$ and $\mathbf{X}[:, j]$, respectively. A sequence of N vectors (a.k.a *tokens*) is denoted by $(\mathbf{x}_n)_{n=1}^N$. We use $[\mathbf{x}_1 | \ldots | \mathbf{x}_N]$ to denote a matrix $\mathbf{X}$ created by stacking $\mathbf{x}_i$s along the columns. We show the inner product between matrices by $\langle \mathbf{X}, \mathbf{Y} \rangle = \sum_{i,j} \mathbf{X}[i, j]\mathbf{Y}[i, j]$ and ℓ_2-norm of a matrix by $\|\mathbf{X}\| = \sqrt{\langle \mathbf{X}, \mathbf{X} \rangle}$.

2. Related Work

CNN-based Segmentation Models. Over the past decade, convolution neural networks (CNNs) with an encoder-decoder architecture, known as U-Net [**?**], have dominated medical image segmentation, achieving state-of-the-art various 2D and 3D various medical image segmentation tasks

Since the introduction of the seminal U-Net [36], CNN-based networks have achieved state-of-the-art results on various 2D and 3D various medical image segmentation tasks [15, 54, 49, 17, 28].

For volume-wise segmentation, tri-planar architectures are sometimes used to combine three-view slices for each voxel, also known for 2.5D methods [28, 29, 46].

In contrast, 3D approaches directly utilize the full volumetric image represented by a sequence of 2D slices or modalities. The intuition of employing varying sizes was followed by multi-scan, multi-path models [24, 25, 8] to capture downsampled features of the image. In addition, to exploit 3D context and to cope with limitation of computational resource, researchers investigated hierarchical frameworks. Some efforts proposed to extract features at multiple scales or assembled frameworks [21].

*Corresponding author. Email: pooya.ashtari@esat.kuleuven.be

Roth et al. [37] proposed a multi-scale framework to obtain varying resolution information in pancreas segmentation. These methods provide pioneer studies of 3D medical image segmentation at multiple levels, which reduces problems in spatial context and low-resolution condition.

Despite their success, a limitation of these networks is their poor performance in learning global context and long-range spatial dependencies, which can severely impact the segmentation performance for challenging tasks.

Vision Transformer. However, convolution operations aggregate only local information in an image, making U-Net generally fail to effectively model long-range dependencies, often necessary for better recognition of lesions. Moreover, convolution kernels are not adaptive in the inference phase in the sense that they are fixed and independent from the input. Hence, this work proposes an effective architecture based on *Transformers* [**?**] and *attention mechanism*, which is able to exploit global context to achieve more accurate segmentation maps.

Matrix Factorization Models. hamburger method

3. Method

3.1. Revisit Attention

The attention mechanism is the key component which enables transformers to model complex dependencies between the elements of a sequence. Consider an input sequence of C-dimensional tokens $(\mathbf{x}_n)_{n=1}^N$, stacked into the rows of matrix $\mathbf{X} = [\mathbf{x}_1|\dots|\mathbf{x}_N]^T$. In a self-attention layer, the input is first projected onto three learnable weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \boldsymbol{W}^V \in \mathbb{R}^{C\times E}$ to get three different matrices: *Query* $\mathbf{Q} = [\mathbf{q}_1|\dots|\mathbf{q}_N]^T = \mathbf{X}\mathbf{W}^Q$, *Key* $\mathbf{K} = [\mathbf{k}_1|\dots|\mathbf{k}_N]^T = \mathbf{X}\mathbf{W}^K$, and *Value* $\mathbf{V} = [\mathbf{v}_1|\dots|\mathbf{v}_N]^T = \mathbf{X}\boldsymbol{W}^V$. The output is then obtained as follows:

$$A(\boldsymbol{X}) = \texttt{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{E}}\right)\boldsymbol{V}, \tag{1}$$

where `softmax` is taken row-wise. Note that attention takes $\mathcal{O}(N^2E)$ time and needs $\mathcal{O}(N^2)$ memory to store the attention map, scaling quadratically with the sequence length, which is prohibitively expensive for large inputs such a high-resolution or 3D images.

Taking a closer look at equation (1), we notice that attention can be viewed as a special case of nonparametric regression. To show this, lets consider key-value pairs $\{(\mathbf{k}_n, \mathbf{v}_n)\}_{n=1}^N$ as a training set and queries $\{\mathbf{q}_m\}_{n=1}^N$ as a test set. Predictions for the queries using a Nadaraya-Watson kernel regression [1] are:

$$\hat{f}(\mathbf{q}_n) = \frac{\sum_{m=1}^N \varphi(\mathbf{q}_n, \mathbf{k}_m)\mathbf{v}_n}{\sum_{m=1}^N \varphi(\mathbf{q}_n, \mathbf{k}_m)}, \tag{2}$$

where kernel $\varphi(.,.)$ is, in general, a similarity measure. Setting $\varphi(\mathbf{a}, \mathbf{b}) = \exp(\mathbf{a}^T\mathbf{b}/\sqrt{E})$ simple yields the attention formula.

3.2. Low-Rank Matrix Factorization

Formulating attention as a kernel regression with softmax kernel suggests that we can probably use other regression methods on queries, keys, and values to model interactions

between the sequence elements. For example, one can take a matrix completion approach by low-rank decomposition of a block matrix of queries, keys, and values with missing entries, that is

$$\begin{pmatrix} \mathbf{K} & \mathbf{V} \\ \mathbf{Q} & ? \end{pmatrix} \approx \begin{pmatrix} \mathbf{B} \\ \mathbf{F} \end{pmatrix} \begin{pmatrix} \mathbf{G} \\ \mathbf{H} \end{pmatrix}^T, \tag{3}$$

where $\mathbf{B}, \mathbf{F} \in \mathbb{R}^{N \times R}$ and $\mathbf{G}, \mathbf{H} \in \mathbb{R}^{E \times R}$; and R in the rank. The reconstructed matrix $\mathbf{F}\mathbf{H}^T$ then gives an estimation of the missing block. This can be viewed as a joint matrix factorization of $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, i.e., $\mathbf{K} \approx \mathbf{B}\mathbf{G}^T$, $\mathbf{V} \approx \mathbf{B}\mathbf{H}^T$, $\mathbf{Q} \approx \mathbf{F}\mathbf{G}^T$.

In this work, we further simplify the procedure by considering only a single linear map to generate a single matrix, e.g., query $\mathbf{Q}$, using a regular factorization rather than a joint one, that is $\mathbf{Q} \approx \mathbf{F}\mathbf{G}^T$, then the output is the reconstructed matrix $\mathbf{F}\mathbf{G}^T$. Depending on the matrix factorization algorithm, a suitable activation function may be applied before the factorization to constrain the input. For example, in the case of nonnegative matrix factorization (NMF), ReLU of the matrix must be first taken to make all of the entries nonnegative.

3.3. Overall Architecture

As shown in Figure 1, a Factorizer model follows a U-Net-style architecture consisting of encoder and decoder parts. Given a 3D input image $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{C_{in} \times H \times W \times D}$ with C_{in} channels and resolution (H, W, D), the network outputs a *logit map* of size (C_{out}, H, W, D), where C_{out} is the number of segmentation classes. A $3 \times 3 \times 3$ convolution layer is used as the stem to increase the number of channels. However, note that in contrast to ViT, Factorizer does not flatten the spatial dimensions to generate a sequence of tokens.

The network has four levels, with the resolution decreasing to 1/16 in the bridge. At each level in the encoder (decoder), the input tensor is downsampled (upsampled) by a factor of two while the number of channels is doubled (halved). Convolution (transposed convolution) with kernel size $(2, 2, 2)$ and stride 2 is used for downsampleing (upsampling). In the bridge, learnable position embeddings are added to the input after downsampling. We use deep supervision [2] at the three highest resolutions in the decoder, applying pointwise convolutions (i.e., convolution with kernel size $(1, 1, 1)$) to get the output and two auxiliary low-resolution logit tensors.

3.4. Factorizer Block

A Factorizer block is constructed by replacing multi-head self-attention (MSA) module in a ViT block by a Wrapped NMF module (described in Section 3.5). As shown in Figure 2a, a Factorizer block comprises NMF module and MLP, each of which follows a LayerNorm and is followed by a residual connection, that is,

$$\begin{aligned} \boldsymbol{\mathcal{Z}}' &= \text{WrappedNMF}(\text{LayerNorm}(\boldsymbol{\mathcal{X}}) + \boldsymbol{\mathcal{X}}, \\ \boldsymbol{\mathcal{Z}} &= \text{MLP}(\text{LayerNorm}(\boldsymbol{\mathcal{Z}}')) + \boldsymbol{\mathcal{Z}}', \end{aligned} \tag{4}$$

where MLP has two linear layers with GELU nonlinearity in between:

$$\text{MLP}(\boldsymbol{\mathcal{Z}}) = \text{PointwiseConv}(\text{GELU}(\text{PointwiseConv}(\boldsymbol{\mathcal{X}})). \tag{5}$$

The number of input and output channels are the same, but the number of inner channels is set twice that of the input channels.

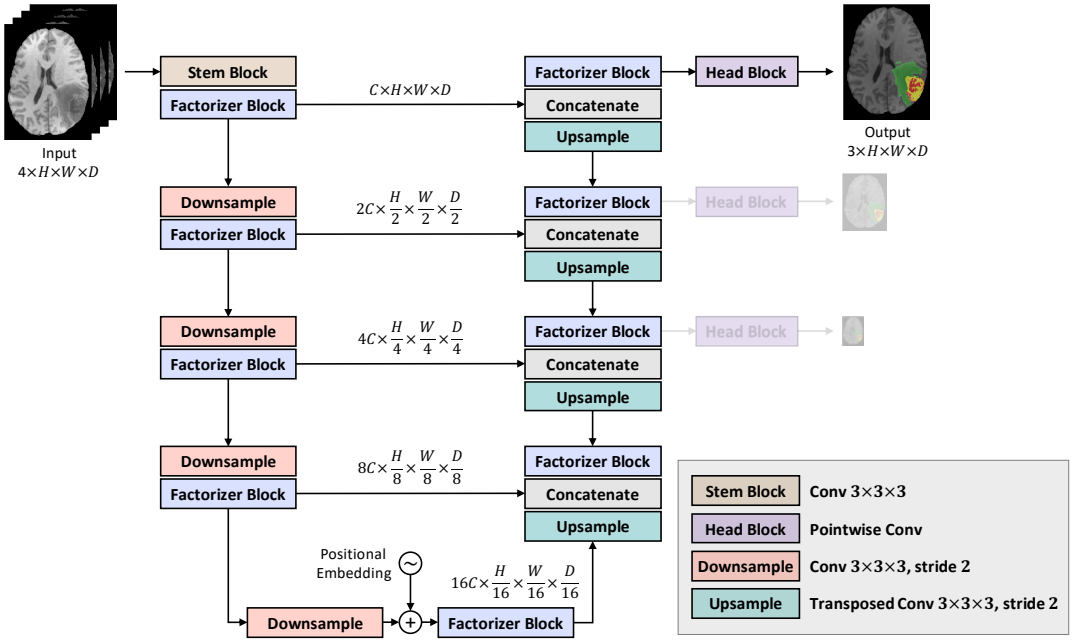


Figure 1 The overall architecture of Factorizer.

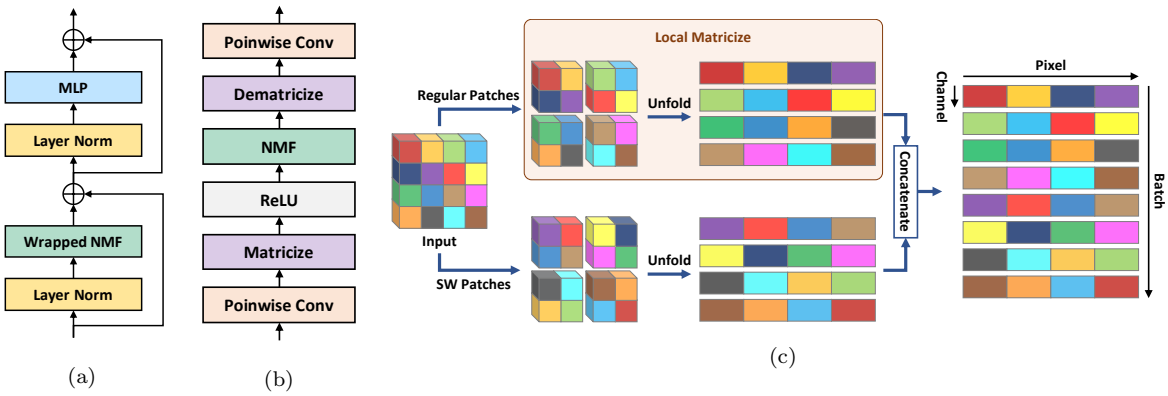


Figure 2 An overview of the Factorizer block and its components. (a) overall Factorizer block (b) Wrapped NMF module (c) An illustration of Swin Matricize on a 2D toy example.

3.5. Wrapped NMF Module

The integral component of Factorizer is Wrapped NMF module, which relies on matricization (i.e., an operation that turns a tensor into a matrix) and nonnegative matrix factorization (NMF). As shown in 2b, a Wrapped NMF subblock first applies a pointwise convolution to linearly project each voxel. The output is then reshaped into a batch of matrices using a *Matricize* operation, which is detailed later on in Section 3.5.1. The resulting matrices pass through ReLU to clamp all their elements into nonnegative values and then low-rank approximated using NMF. The constructed matrices are reshaped back to their original size using the *Dematricize* operation, i.e., the inverse of Matricize. Finally, another pointwise convolution is applied, yielding the output. More formally,

Wrapped NMF can be computed as follows:

$$\begin{aligned}
\boldsymbol{\mathcal{X}}^1 &= \text{PointwiseConv}(\boldsymbol{\mathcal{X}}),\\
\boldsymbol{\mathcal{X}}^2 &= \text{Matricize}(\boldsymbol{\mathcal{X}}^1),\\
\boldsymbol{\mathcal{X}}^3 &= \text{NMF}(\text{ReLU}(\boldsymbol{\mathcal{X}}^2)),\\
\boldsymbol{\mathcal{X}}^4 &= \text{Dematricize}(\boldsymbol{\mathcal{X}}^3),\\
\text{WrappedNMF}(\boldsymbol{\mathcal{X}}) &= \text{PointwiseConv}(\boldsymbol{\mathcal{X}}^4),
\end{aligned} \tag{6}$$

where intermediate tensors $\boldsymbol{\mathcal{X}}^i$s and the output have the same size as that of input $\boldsymbol{\mathcal{X}}$.

3.5.1. Matricize

Before applying any matrix factorization method, an input batch of multi-dimensional images, denoted by $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B\times C\times H\times W\times D}$, must be turned into a batch of matrices, say $\mathbf{Z} \in \mathbb{R}^{B'\times M\times N}$. This operation is called *Matricize*. In this work, we propose three Matricize operation: i) Global Matricize, ii) Local Matricize, and iii) Shifted Window (SW) Matricize. Depending on which operation is used, different variants for Factorizer are obtained: i) Global Factorizer, ii) Local Factorizer, and iii) **S**hifted **Win**dow (Swin) Factorizer. The Matricize operations are described in the following.

Global Matricize This operation simply flattens the spatial dimensions and also divides the channels into multiple groups (analogous to heads of Multi-Head Self-Attention). More specifically, Global Matricize reshapes a batch of C-channel 3D images $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B\times C\times H\times W\times D}$ into a batch of matrices denoted by a 3D tensor $\mathbf{Z} \in \mathbb{R}^{B(C/E)\times E\times HWD}$, where E is called the head dimension, i.e., the number of channels per head (or matrix). This operation is obviously suitable for modeling global context and imposes no locality inductive bias.

Local Matricize Global Matricize lacks a notion of locality which is typically useful for images especially in low-data regime. Local Matricize is proposed to mitigate this shortcoming by splitting an input $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B\times C\times H\times W\times D}$ into a grid of non-overlapping patches of size (E, P, P, P). These patches are flattened spatially and then concatenated along the batch dimension, yielding a batch of matrices presented by the tensor $\mathbf{Z} \in \mathbb{R}^{BCHW/(EP^3)\times E\times P^3}$. The entire procedure can be summarized by Einstein notation:

$$\mathbf{Z}[bg_cg_hg_wg_d, e, p_ep_wp_d] = \boldsymbol{\mathcal{X}}[b, g_ce, g_hp_h, g_wp_w, g_dp_d] \tag{7}$$

where (g_c, g_h, g_w, g_d) and (e, p_h, p_w, p_d) indices correspond to grid and patch dimensions, respectively. In PyTorch and TensorFlow, such Einstein operations can be simply implemented using `rearrange` function from `einops` library. Once NMF is applied to the resulting batch of matrices, the inverse operation of Local Matricize, called Local Dematricize, must be applied to transform them back to the initial shape. Local Dematricize can be easily obtained by composing the inverses of sub-operations in reverse order. In practice, Local Dematricize can also be formulated and implemented as an Einstein operation. A PyTorch-style pseudocode of Local (De)matricize module is provided in Algorithm 3.

Note that while Local Matricize seems to reshape an image in a similar way to the input patchifier of ViT, it concatenates patches along batch dimension rather than channel dimension. In fact, Local Matricize can be used when modeling within-patch interactions is desirable, which is different form ViT-based approaches, where interactions between embedded patches are typically modeled.

Shifted Window Matricize. While Local Matricize introduce locality to a model, voxels close to the boundaries of partitioning windows are not represented effectively. Since patches are low-rank approximated independently later on in an NMF layer, two neighboring boundary voxels from two adjacent patches are very likely to end up having excessively different feature maps in the output of the factorizer block, which in turn eventually makes the prediction less accurate for such voxels. To mitigate this problem, we utilize a shifted window approach proposed by Liu et al. [3] and introduce a Shifted Window (SW) Matricize operation, making the output feature maps smoother around boundaries.

An illustration of SW Matricize is provided in Figure 2c. Here, in addition to regular patches similar to those extracted by Local Matricize, shifted window (SW) patches are also included. Let's $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B \times C \times H \times W \times D}$ be the input and (P, P, P) the patch size. To extract SW patches, the input must be first shifted by the offset of $(P/2, P/2, P/2)$ along spatial dimensions such that voxels shifted beyond the boundaries of the images are re-introduced at the first position, yielding a tensor of the same size $B \times C \times H \times W \times D$. We call this operation *Roll* (which can be implemented using `roll` function in PyTorch and TensorFlow). The resulting rolled tensor is then reshaped by Local Matricize to get SW patches. Finally, both the batches of regular and SW patches are concatenated along the batch dimension. Formally, SW Matricize are computed as follows:

$$\begin{aligned} \boldsymbol{\mathcal{X}}^1 &= \text{LocalMatricize}_P(\boldsymbol{\mathcal{X}}), \\ \boldsymbol{\mathcal{X}}^2 &= \text{LocalMatricize}_P(\text{Roll}_{P/2}(\boldsymbol{\mathcal{X}})), \\ \text{SWMatricize}(\boldsymbol{\mathcal{X}}) &= \text{Concatenate}(\boldsymbol{\mathcal{X}}^1, \boldsymbol{\mathcal{X}}^2), \end{aligned} \tag{8}$$

where LocalMatricize_P denotes Local Matricize with a patch size of (P, P, P), and $\text{Roll}_{P/2}$ denotes Roll operator with a shift $(P/2, P/2, P/2)$. To build SW Dematricize, we need to reconstruct the image from both regular and SW patches independently then compute their average to achieve a smoother and more accurate feature maps. Further details are provided in Appendix B, which includes a PyTorch implementation of SW (De)matricize module presented in Algorithm 3.

3.5.2. Nonnegative Matrix Factorization

Once an input is somehow transformed to a batch of matrices, and its negative elements are clipped to zero by ReLU, it is ready to be low-rank approximated by Nonnegative Matrix Factorization (NMF). This is the main component of a factorizer model that contributes most to modeling local or global context in an image.

NMF [4] seeks to approximate some given nonnegative matrix $\mathbf{X} \in \mathbb{R}_{\geq 0}^{M \times N}$ by

$$\mathbf{X} \approx \mathbf{F}\mathbf{G}^T, \tag{9}$$

where $\mathbf{F} \in \mathbb{R}_{\geq 0}^{M \times R}$ and $\mathbf{G} \in \mathbb{R}_{\geq 0}^{N \times R}$ are factors, and the positive integer $R \leq \min(M, N)$ is the rank. Once the factors $\mathbf{F}$ and $\mathbf{G}^T$ are somehow approximated, the NMF layer in a Factorizer block outputs the reconstructed matrix $\hat{\mathbf{X}} = \mathbf{F}\mathbf{G}^T$. Various loss functions have been used to form the objective function and measure the quality of an approximation. Depending on the loss function, constraints, and regularization, many variants of NMF have been proposed. In this work, we use a standard NMF with the squared error, which is the most-widely used variant for images, to find factors by solving

$$\underset{\mathbf{F} \geq 0, \mathbf{G} \geq 0}{\text{minimize}} \; \|\mathbf{X} - \mathbf{F}\mathbf{G}^T\|^2. \tag{10}$$

Algorithm 1: Multiplicative update solver for NMF ($\circ$ denotes element-wise matrix product).

Input: $\mathbf{X} \in \mathbb{R}_{\geq 0}^{M \times N}$ (input matrix), R (rank); T (# iterations)
Output: $\hat{\mathbf{X}} \in \mathbb{R}_{\geq 0}^{M \times N}$ (low-rank approximated matrix)

Initialize factors $\mathbf{F} \in \mathbb{R}_{\geq 0}^{M \times R}$ and $\mathbf{G} \in \mathbb{R}_{\geq 0}^{N \times R}$
for $t = 1, \ldots, T$ **do**
  $\mathbf{F} \leftarrow \mathbf{F} \circ \dfrac{\mathbf{X}\mathbf{G}}{\mathbf{F}\mathbf{G}^T\mathbf{G}}$
  $\mathbf{G} \leftarrow \mathbf{G} \circ \dfrac{\mathbf{X}^T\mathbf{F}}{\mathbf{G}\mathbf{F}^T\mathbf{F}}$
return $\hat{\mathbf{X}} = \mathbf{F}\mathbf{G}^T$

Algorithm 2: Hierarchical alternating least squares for NMF.

Input: $\mathbf{X} \in \mathbb{R}_{\geq 0}^{M \times N}$ (input matrix), R (rank); T (# iterations)
Output: $\hat{\mathbf{X}} \in \mathbb{R}_{\geq 0}^{M \times N}$ (low-rank approximated matrix)

Initialize factors $\mathbf{F} \in \mathbb{R}_{\geq 0}^{M \times R}$ and $\mathbf{G} \in \mathbb{R}_{\geq 0}^{N \times R}$
for $t = 1, \ldots, T$ **do**
  $\mathbf{A} = \mathbf{X}\mathbf{G}$, $\mathbf{B} = \mathbf{G}^T\mathbf{G}$
  for $r = 1, \ldots, R$ **do**
    $\mathbf{F}[:, r] \leftarrow \max(0, \dfrac{\mathbf{A}[:, r] - \sum_{\ell \neq r} \mathbf{B}[\ell, r]\mathbf{F}[:, \ell]}{\|\mathbf{G}[:, r]\|^2})$
  $\mathbf{A} = \mathbf{X}^T\mathbf{F}$, $\mathbf{B} = \mathbf{F}^T\mathbf{F}$
  for $r = 1, \ldots, R$ **do**
    $\mathbf{g}[:, r] \leftarrow \max(0, \dfrac{\mathbf{A}[:, r] - \sum_{\ell \neq r} \mathbf{B}[\ell, r]\mathbf{G}[:, \ell]}{\|\mathbf{F}[:, r]\|^2})$
return $\hat{\mathbf{X}} = \mathbf{F}\mathbf{G}^T$

In general, problem (10) is nonconvex, and finding global minima is NP-hard. However, numerous iterative algorithms have been proposed to find a "good local minimum". The majority of existing methods are based on a block coordinate descent (BCD) scheme (a.k.a alternating optimization), where the objective function is iteratively minimized with respect to one factor while the other factor is kept fixed. That is, the convex subproblems

$$\mathbf{F} \leftarrow \arg\min_{\mathbf{F} \geq 0} \|\mathbf{X} - \mathbf{F}\mathbf{G}^T\|^2, \qquad \mathbf{G} \leftarrow \arg\min_{\mathbf{G} \geq 0} \|\mathbf{X}^T - \mathbf{G}\mathbf{F}^T\|^2, \tag{11}$$

are exactly or approximately solved alternately.This ensures that the objective function value does not increase after each update and guarantees convergence to a stationary point under some mild conditions [5].

Among numerous BCD-based algorithms for NMF, Multiplicative Update (MU) [4] is the best-known due to the advantage of being easy-to-implement and scalable. MU enforces nonnegativity constraint by updating the previous values of a factor matrix by multiplication with a nonnegative scale factor. A pseudocode of MU is pretended in Algorithm 1. However, slow convergence of MU has been pointed [6], and hence

more effective algorithms, such as Hierarchical Alternating Least Squares (HALS) [7], with faster convergence have been introduced. HALS updates a factor matrix, say $\mathbf{F} = [\mathbf{f}_1|\dots|\mathbf{f}_R]$, with inner iterations, in which the columns $\mathbf{f}_r$s are updated by solving the following subproblem

$$\mathbf{f}_r \leftarrow \underset{\mathbf{f}_r \geq 0}{\arg\min} \, \|\mathbf{E}_r - \mathbf{f}_r \mathbf{g}_r^T\|^2, \tag{12}$$

where $\mathbf{E}_r = \mathbf{X} - \sum_{\ell \neq r}^{R} \mathbf{f}_\ell \mathbf{g}_\ell^T$ is the residual matrix, which is, in fact, approximated by a rank-one matrix. An encouraging aspect of HALS is that each subproblem (12) can be easily shown to have a closed-form solution:

$$\mathbf{f}_r^* \leftarrow \max(0, \frac{\mathbf{E}_r \mathbf{g}_r}{\|\mathbf{g}_r\|^2}). \tag{13}$$

Similarly, the update formula for columns of $\mathbf{G}$ can be derived. Note that HALS is a $2R$-block coordinate descent procedure, where at each outermost iteration, first the columns of $\mathbf{F}$ and then the columns of $\mathbf{G}$ are updated. Algorithm 2 provides a pseudocode of HALS (further details on MU and HALS can be found in [6], Chapter 8).

A special case of NMF is $R = 1$; i.e., $\mathbf{X} \approx \mathbf{f}\mathbf{g}^T$, where $\mathbf{f} \in \mathbb{R}_{\geq 0}^M$ and $\mathbf{g} \in \mathbb{R}_{\geq 0}^N$; for which one can easily investigate that both MU and HALS are simplified to the same update rule:

$$\mathbf{f} \leftarrow \frac{\mathbf{X}\mathbf{g}}{\|\mathbf{g}\|^2}, \qquad \mathbf{g} \leftarrow \frac{\mathbf{X}\mathbf{f}}{\|\mathbf{f}\|^2}. \tag{14}$$

In this paper, all Factorizer models are trained with $R = 1$, and compression ratios (and somewhat reconstruction errors) are controlled by adjusting the head dimension (i.e., the number of rows in a matrix, as discussed in 3.5.1) to sufficiently small values. This means that a matricize operation transforms an image to a batch of such fat matrices (whose number of columns are far grater than that of its rows) that rank-one approximation would suffice in practice. This greatly simplifies a factorizer model and improve the interpretability while yielding better performance in segmentation. However, in one of our ablation studies (see Section 4.5), we experimented with both MU and HALS for $R > 1$ to investigate the impact of rank in the inference phase.

It is worth mentioning that not all NMF algorithms and their settings can be used in the NMF layer of a Factorizer block. The utilized algorithm must have some properties so that we can ultimately train the Factorizer model successfully in an end-to-end fashion using a gradient descent-based optimizer on GPU(s) in an existing deep learning framework, such as PyTorch. Firstly, the algorithm should be backpropagation-friendly and amenable to automatic differentiation, that is $\frac{\partial \hat{\mathbf{X}}}{\partial \mathbf{X}}$ should not only be well-defined and somewhat smooth but also computable by an existing deep learning frameworks, such as PyTorch, so that we can practically train the Factorizer model in an end-to-end fashion using a gradient descent optimizer. Another related aspect is that the gradient $\frac{\partial \hat{\mathbf{X}}}{\partial \mathbf{X}}$, as explained in [8], stats to vanish during backpropagation after some iterations. Therefore, the number of outer iterations T in an NMF algorithm should be limited in order to have stable gradients. For MU and HALS, $T = 5$ is a reasonable choice in practice. Finally, update rules should be also friendly to GPU parallel processing for exploiting GPUs to train Factorizers in a reasonable amount of time. Taking all these factors into account, MU and HALS are appropriate choices. While HALS has better convergence properties, MU is more favorable for GPU training due to unparallelizable inner iterations of HALS.

4. Experiments

4.1. dataset.

We evaluate the effectiveness of our models in segmenting brain tumors on the Brain Tumour Segmentation (BraTS) dataset [9, 10, 11] from Medical Segmentation Decathlon (MSD) (Task 1) [12]. The BraTS training set of consists of 484 multiparametric MRI (mpMRI) scans from patients diagnosed with either low-grade glioma or high-grade glioma (glioblastoma). Each scan comes with four 3D MRI sequences, namely T2 Fluid-Attenuated Inversion Recovery (FLAIR), native T1-weighted (T1), post-Gadolinium contrast T1-weighted (T1gd), and T2-weighted (T2). Once images are prepossessed;i.e., rigidly co-registered the same anatomical template, resampled to the same voxel spacing 1mm^3, and skull-stripped; the ground truths are manually created by experts who label each voxel as enhancing tumor (ET), edema (ED), necrotic and non-enhancing tumor (NCR/NET), or everything else. However, for evaluation, the 3 nested subregions, namely enhancing tumor (ET), tumor core (TC–i.e., the union of ED and NCR/NET), and whole tumor (WT) are used (see the sample ground truths in Figure 5)

4.2. Setup

All the models were implemented using PyTorch [13] and MONAI [14] frameworks and trained on NVIDIA P100 GPUs. We followed the same training workflow in all the experiments. In the following, we first provide the details of this workflow and baseline models, then present the evaluation protocol and the results.

Preprocessing. For each scan in the BraTS dataset, a 4-channel 3D image as the input was first constructed by concatenated the four modalities, FLAIR, T1, T1gd, and T2. The image and its ground truth were then cropped with a minimal box filtering out zero regions. The image was normalized channel-wise using z-score to have intensities with zero mean and unit variance. Random patches of size $(128, 128, 128)$ were extracted from the images for training. To reduce overfitting, we used data augmentation techniques, including random affine transform, random flip along each spatial dimension, additive Gaussian noise, random Gaussian smoothing, random scale and shift intensity, and random gamma intensity transform. The further details are provided in Appendix A.

Training. All models were trained for 100k steps with batch size 2 (one sample per GPU) using AdamW optimizer with a base learning rate of 1e−4, weight decay of 1e−2, warmup of 2k steps, and cosine annealing scheduler. The loss function is a combination of *soft Dice loss* [15] and cross-entropy loss, defined as

$$\mathcal{L}(\mathbf{G}, \mathbf{Y}) = \mathcal{L}_{\text{Dice}}(\mathbf{G}, \mathbf{Y}) + \mathcal{L}_{\text{CE}}(\mathbf{G}, \mathbf{Y}), \tag{15}$$

where

$$\mathcal{L}_{\text{Dice}}(\mathbf{G}, \mathbf{P}) = 1 - \frac{2\langle \mathbf{G}, \mathbf{Y} \rangle}{\|\mathbf{G}\|^2 + \|\mathbf{Y}\|^2}, \qquad \mathcal{L}_{\text{CE}}(\mathbf{G}, \mathbf{Y}) = -\frac{1}{N}\langle \mathbf{G}, \log(\mathbf{Y}) \rangle, \tag{16}$$

where $\mathbf{G} \in \{0, 1\}^{J \times N}$ and $\mathbf{Y} \in [0, 1]^{J \times N}$ represent the one-hot encoded ground truth and the predicted probability map for each voxel, respectively, with J denoting the number of segmentation classes and N denoting the number of voxels in the patch. The three deep supervision outputs and the corresponding downsampled ground truths are used for computing the final loss. The training objective is the weighted mean of the losses at all

these resolutions, with the weights being 1, 0.5, and 0.25 at resolutions 128^3, 64^3, and 32^3, respectively.

Inference. A test image in the inference was first subjected to z-score intensity normalization, then the prediction was made using a sliding window approach with a 50% overlap and a window size of $128 \times 128 \times 128$ (same as the patch size used in training). Finally, the resulting probabilities were thresholded by 0.5 to obtain a binary segmentation map.

Evaluation Metrics. The Dice score and Hausdorff Distance 95% (HD95) were used as metrics to asses the performance of models in our experiments. For each segmentation region, Dice score measures the voxel-wise overlap between the ground truth and the prediction, defined as

$$\text{Dice}(\mathbf{g}, \mathbf{p}) = \frac{2 \sum_{n=1}^{N} g_n p_n}{\sum_{n=1}^{N} g_n + \sum_{n=1}^{N}, p_n} \tag{17}$$

where $g_n \in \{0, 1\}$ and $p_n \in \{0, 1\}$ represent the ground truth and the binary prediction for a voxel, respectively, and N is the number of voxels. If both the ground truth and the prediction do not have any nonzero values, that is the denominator of equation (17) is zero, the Dice score is defined as 1. Hausdorff Distance (HD) evaluates the distance between the boundaries of ground truth and prediction. HD is defined as follows:

$$\text{HD}(G, P) = \max\{\max_{\mathbf{g} \in G} \min_{\mathbf{p} \in P} \|\mathbf{g} - \mathbf{p}\|, \max_{\mathbf{p} \in P} \min_{\mathbf{g} \in G} \|\mathbf{p} - \mathbf{g}\|\}, \tag{18}$$

where G and P denote the set of all voxels on the surface of ground truth and prediction, respectively. HD95 is a more robust version to outliers, which calculates the 95% quantile rather than the maximum of surface distances.

Models. In all the experiments, the number of output channels of the stem block was 32. The patch size of Local and SW Matricize was set to $(8, 8, 8, 8)$. In NMF modules, the factor matrices were initialized with uniform distribution $\mathcal{U}(0, 1)$. In training, we used a rank-one approximation ($R = 1$) with $T = 5$ outer iterations of HALS (which is equivalent to MU for $R = 1$), as described in Section 3.5.2.

To make a fair comparison between factorizer and baseline models, we neutralize the effect of architectural variability by keeping the overall architecture of baselines same as that of Factorizer and only changing encoder and decoder blocks. This helps to better investigate the impact of blocks rather than architectures. Three models are used as baselines, all of which follow the same overall architecture illustrated in Figure 1, except that each has a different encoder and decoder block. The baselines are detailed in the following:

- U-Net: This model is based on 3D U-Net [16] with minor modifications. Each encoder (and decoder) block is composed of two convolutions with a kernel size of $(3, 3, 3)$. Each convolutional layer is followed by LeakyReLU nonlinearity and Group Normalization (with a group size of 8) in order. This model does not have any positional embedding in its bridge since CNNs already have some notion of position. This model is very similar to nnU-Net [17]

- Res-U-Net: ResNet block [18] is the cornerstone of Res-U-Net. This block is similar to that of U-Net, except that it has residual connection after the last Group Normalization. This model is very similar to SegResNet [19].

Table 1 Comparison of different models on the BraTS dataset. Scores are obtained by 5-Fold cross-validation. The asterisks indicate how significantly our models scores differ from those of baselines according to pairwise Wilcoxon signed-rank test (*: p-value < 0.05; **: p-value < 0.01; ***: p-value < 0.001; and ****: p-value < 0.0001).

Model	#Params	FLOPs	Dice (%)				Hausdorff 95% (mm)			
			ET	TC	WT	Avg.	ET	TC	WT	Avg.
U-Net	28.7M	1264.7G	75.89	81.94	90.09	82.64	8.61	9.13	9.31	9.19
Res-U-Net	28.9M	1280.8G	77.95	82.49	**90.39***	83.61	6.08	8.18	**8.06****	7.52
Performer	6.7M	321.8G	78.43	82.34	88.72	83.16	6.72	9.78	12.93	10.21
Global Factorizer	5.9M	282.2G	78.20	82.86	88.65	83.24	6.67	9.53	11.69	9.71
Local Factorizer	5.9M	283.9G	78.67	82.85	89.30	83.61	5.29	7.77	8.91	7.41
Swin Factorizer	5.9M	290.4G	**79.33**	**83.14*****	90.16	**84.21****	**4.91****	**7.31***	8.23	**6.89***

- Performer: The encoder and decoder blocks of this model are based on ViT (note that an input is first flattened into a sequence of voxels before feeding to a ViT block). However, attention scales quadratically with the number of voxels, hence is prohibitively expensive in our case, where an input can have up to 128^3 voxels. Therefore, we replace original attention layers of a ViT block with FAVOR+ (Fast Attention Via positive Orthogonal Random features) used in Performer, recently proposed by Choromanski et al. [20] as a linearly scalable alternative to Transformer. FAVOR+ gives an unbiased estimate of attention using only linear (as opposed to quadratic) time and space complexity. Note that except for the attention layers, the rest of the components of this baseline is same as those of Factorizer.

4.3. Results

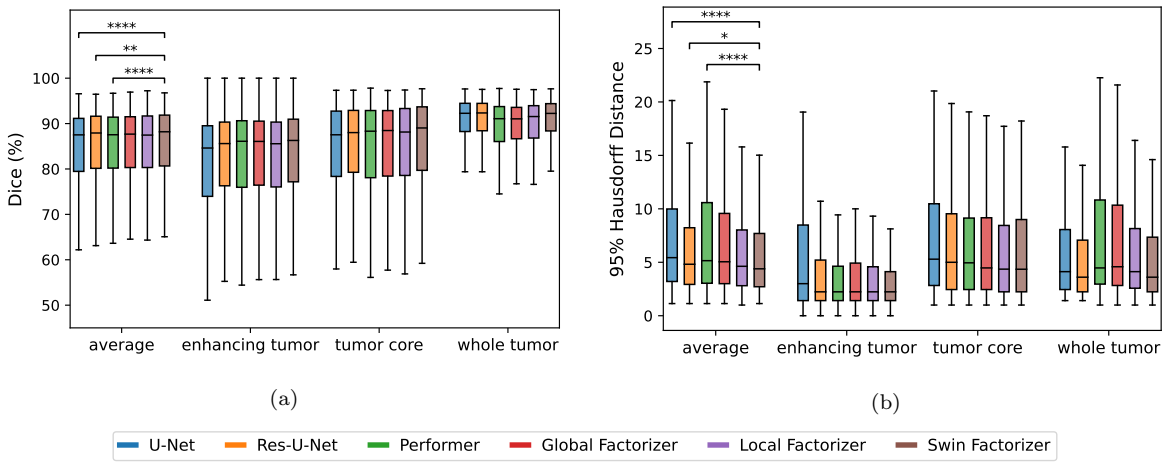
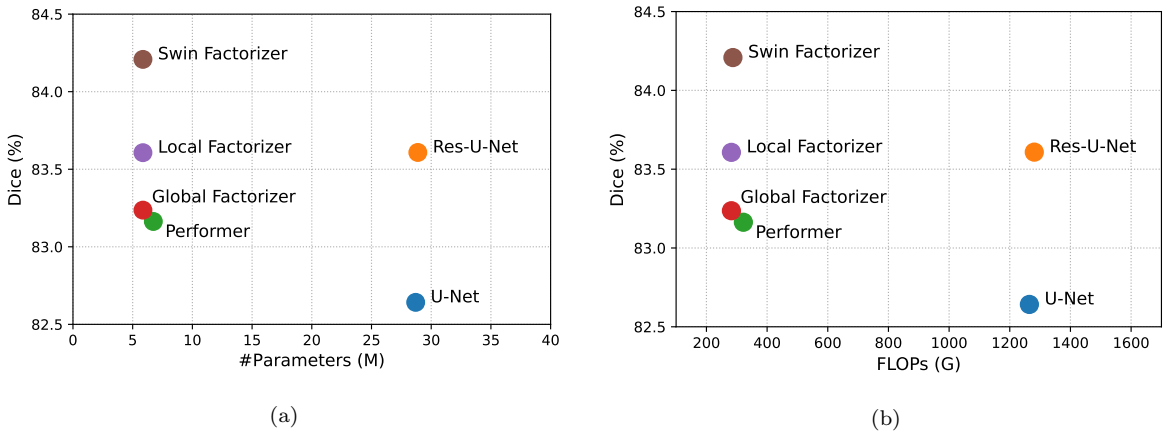


Figure 3 The Dice score and 95% Hausdorff distance of different models on the BraTS datasest. The asterisks indicate statistical significance according to pairwise Wilcoxon signed-rank test (*: p-value < 0.05; **: p-value < 0.01; ***: p-value < 0.001; and ****: p-value < 0.0001).

Quantitative Evaluation. For all the experiments, we performed 5-fold cross-validation to estimate how capable our models are in generalizing to unseen data. Pairwise Wilcoxon signed-rank test was also for comparing the performance of our models with that of the

baselines. The results on the BraTS dataset are reported in Table 1 and illustrated by box plots in Figure 3.

Swin Factorizer was superior to all the other models, which achieved an average Dice of 84.21% and HD95 of 6.89, significantly outperforming Res-U-Net, the best baseline, with p-values of < 0.01 for average Dice and < 0.05 for average HD95, while requiring substantially less computations. Swin Factorizer was the best-performing on ET and TC subregions, particularly yielding a Dice of 83.14% on TC, which is significantly higher than the others with p-value < 0.001. Local Factorizer was our second best model, and its performance ws comparable with that of Res-U-Net. More specifically, Local Factorizer outperformed Res-U-Net on ET and TC while underperformed on WT. As shown in Figure 4, while achieving competitive performance over U-Net and Res-U-Net, Factorizer models have much fewer parameters and are significantly cheaper, taking over four times lower theoretical computational cost (FLOPs).



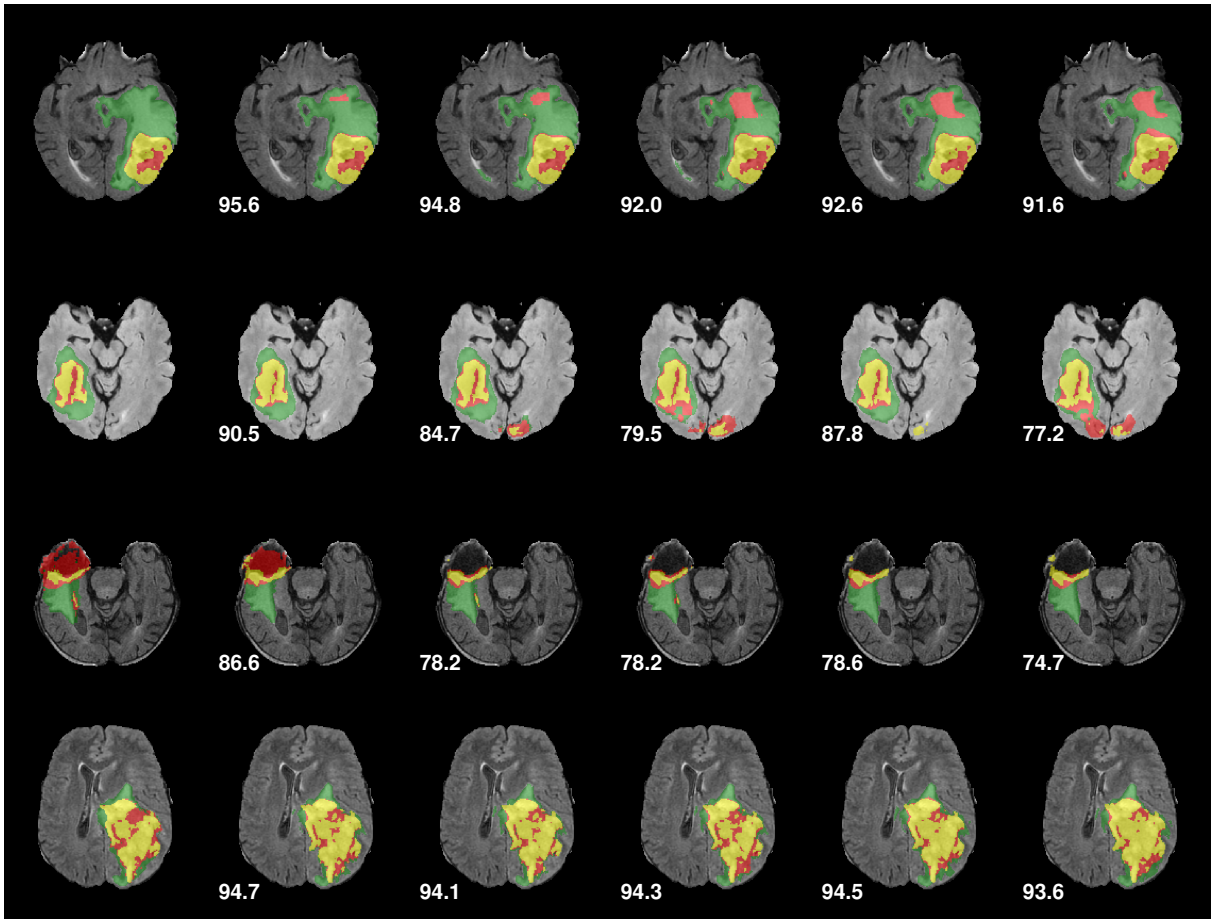
(a) (b)

Figure 4 Performance of Factorizer models in terms of average Dice on BraTS, versus their speed in terms of the number of FLOPs (left) and versus the number of parameters (right).

Performer is the transformer-based counterpart of Global Factorizer in the sense that they both only exploit global context without imposing any locality inductive bias in their blocks. However, while having lower computational complexity, Global Factorizer, with an average Dice score of 83.24% and an average HD95 of 9.71, marginally outperformed Performer, with an average Dice score of 83.16% and an average HD95 of 10.21. Local Factorizer improved the performance of Global Factorizer by exploiting locality, particularly with the average HD95 significantly dropping from 9.71 to 7.41 (p-value < 0.0001). As expected, Swin Factorizer improved all the scores of Local Factorizer, which is consistent with the fact that Swin Factorizer modifies Local Factorizer by better representing the boundary voxels in Local Matricize.

Qualitative Comparisons. Qualitative comparisons of glioma segmentation models are presented in Figure 5. Swin Factorizer demonstrates superior performance in segmenting TC and ET. This capability is evident in row 1 (row 2), where Swin Factorizer more successfully delineates TC compared to the other models, which do not as accurately distinguish edema (normal tissues) from tumor core. Particularly, the CNN-based models, i.e., U-Net and Res-U-Net, missclassify a significantly larger part of edema (normal tissues) as necrosis (enhancing tumor). Row 3 exemplify a successful detection of TC by Swin Factorizer, while the other models miss a fairly large necrotic region.

In comparison to Performer (the transformer-based model), Global Factorizer exhibits



Ground Truth **Swin Factorizer** **Global Factorizer** **Performer** **Res-U-Net** **U-Net**

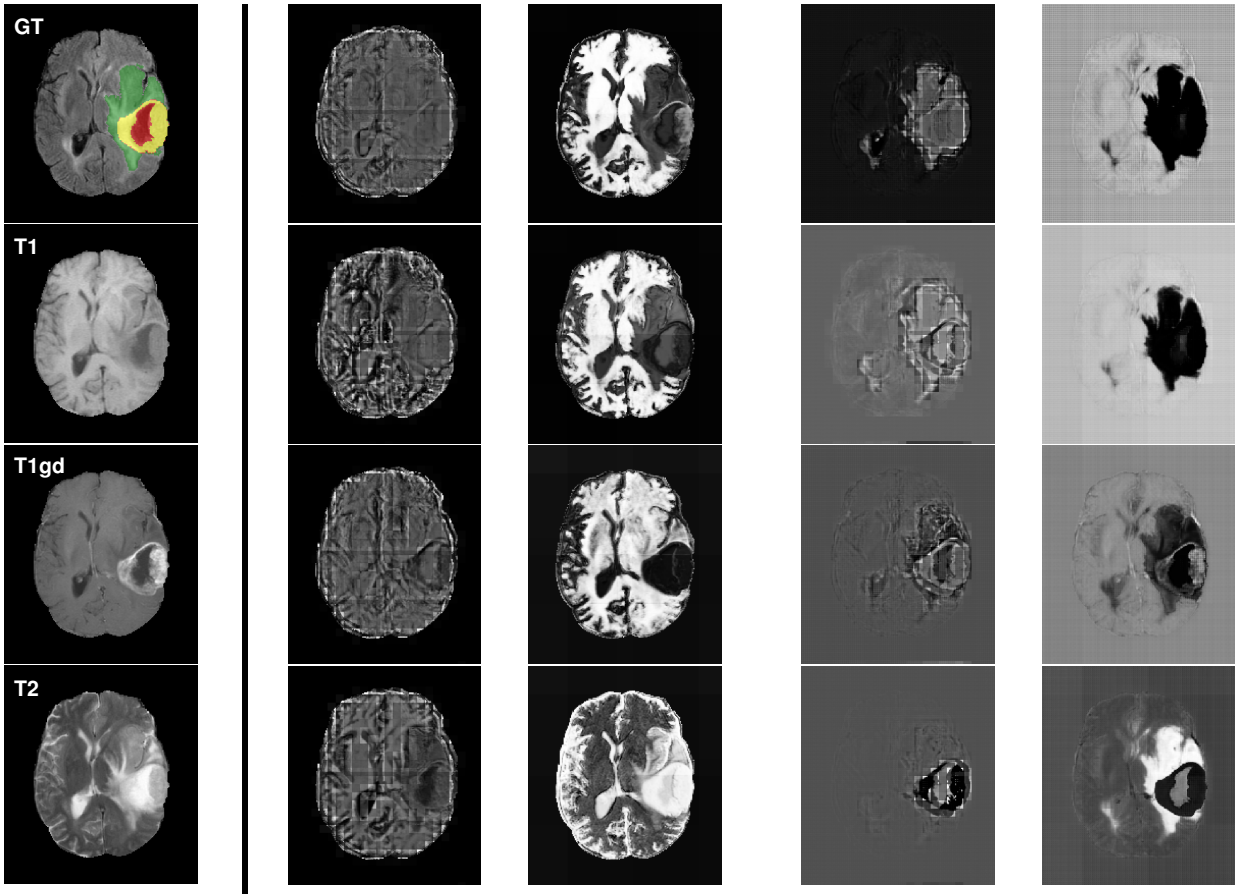
Figure 5 Qualitative results on brain tumor segmentation. ET is denoted in yellow, TC is the union of red and yellow regions, and WT includes the union of green, red, and yellow regions. The average Dice score is presented for each subject.

slightly higher accuracy in identifying tumor cores, especially differentiating necrosis from edema or normal regions, as seen in row 1 and 2. As expected, compared to Global Factorizer, Swin Factorizer demonstrates higher level of ability to capture the fine-grained details of enhancing tumor and necrotic regions.

4.4. NMF Components Interpretability

One additional advantage of Factorizer over Transformer and CNN models is its higher interpretability resulting from meaningful components of NMF. Note that both the fist and last Factorizer blocks have $C = 32$ channels, divided into groups of 8-channel heads during matricization, where the NMF of each head has only a single rank-one term, i.e., $R = 1$. As a results, in total, there are $CR/8 = 4$ components in both the first and last blocks. Figure 6b and 6c show the components of first and last NMF layers on a high-grade glioma case for Swin Factorizer and Global Factorizer. More precisely, each component illustrates the factor matrix corresponding to spatial dimensions after dematricization.

Interestingly, the components are highly meaningful and interpretable in the sense that each component gives an interpretation by differentiating one region from another. For instance, as observed in Figure 6b, the first (row 1) and second (row 2) compo-



(a) Modalities (b) Components of first NMF layer (c) Components of last NMF layer

Figure 6 NMF Components of Factorizer models on BraTS. Panel (a) shows the modalities and ground truth on for a high-grade glioma case. Panel (b) and (c) present the NMF spatial components in first and last block, receptively.

nents discriminate roughly the whole tumor while the third (row 3) and fourth (row 4) components capture the tumor core. For Swin Factorizer, necrosis is distinguished more clearly in the third and fourth components whereas for Global Factorizer necrosis is more recognizable in the second component. As we get to deeper layers, components become even more meaningful such that in the last layer, each component clearly detect some regions. For example, as seen in Figure 6c, the first (row 1) and second (row 2) components very clearly discriminate the whole tumor while the third (row 3) and fourth (row 4) components also capture the tumor core and necrosis.

Another interesting observation is that Global Factorizer yield more discriminative and higher-level components in the first layer, which can be attributed to the fact Global Factorizer models long-range dependencies through all of its blocks, including the first one; however, the receptive field of Swin Factorizer in the first stage of encoder is small but progressively increases as it passes through downsampling layers. Therefore, Swin Factorizer, similar to CNNs, extracts lower-level features in shallower layers and higher-level ones in deeper layers. Notice that the footprints of sliding windows, which appear as a grid pattern, are also evident in all the components of Swin Factorizer,

4.5. Ablation Studies

4.6. Ablation Studies in Training

We conducted ablation studies on Factorizer to further investigate the effects of Factorizer subblocks and positional embedding. In this section, models with an ablated layer were trained from scratch on the BraTS dataset.

Factorizer Subblock. Ablation results of sublocks on brain tumor segmentation are reported in Table 2. When we removed NMF subblocks from a Factorizer model, the performance substantially dropped while Factorizer models without MLP subblocks demonstrated lesser deterioration in results. Local Factorizer without MLP blocks yielded an average Dice of 82.67%, still outperforming U-Net, and Swin Factorizer without MLP blocks yielded an average Dice of 83.57%, which is significantly greater than that of U-Net and comparable with that of Res-U-Net. These results indicate the effectiveness of NMF layer in improving the models.

Table 2 The impact of each sublock on the performance of Factorizer models on BraTS.

MLP Subblock	NMF Subblock	Matricize	Dice (%)				Hausdorff 95% (mm)			
			ET	TC	WT	Avg.	ET	TC	WT	Avg.
✓	✗	✗	77.02	80.89	87.36	81.76	6.55	8.90	11.29	9.02
✗	✓	Global	77.51	80.78	87.85	82.05	8.82	11.24	16.18	12.33
✗	✓	Local	77.79	81.67	88.56	82.67	5.50	8.67	9.44	8.04
✗	✓	SW	78.60	82.61	89.51	83.57	5.00	7.47	7.86	6.87
✓	✓	SW	79.33	83.14	90.16	84.21	4.91	7.31	8.23	6.89

Positional Embedding. with vs without positional embedding

Table 3 The impact of positional embedding.

Model	Positional Embedding	Dice (%)				Hausdorff 95% (mm)			
		ET	TC	WT	Avg.	ET	TC	WT	Avg.
Global Factorizer	✗								
Global Factorizer	✓								
Local Factorizer	✗								
Local Factorizer	✓								
Swin Factorizer	✗								
Swin Factorizer	✓								

4.7. Ablation Studies in Inference

Since the output of an NMF layer is a low-rank approximation of the input, it makes sense to perform an ablation study in the inference phase, for instance by short-circuiting the NMF layer. For all the experiments of this section, we ablated some NMF layers or change their settings in the inference phase after training the model on the BraTS dataset.

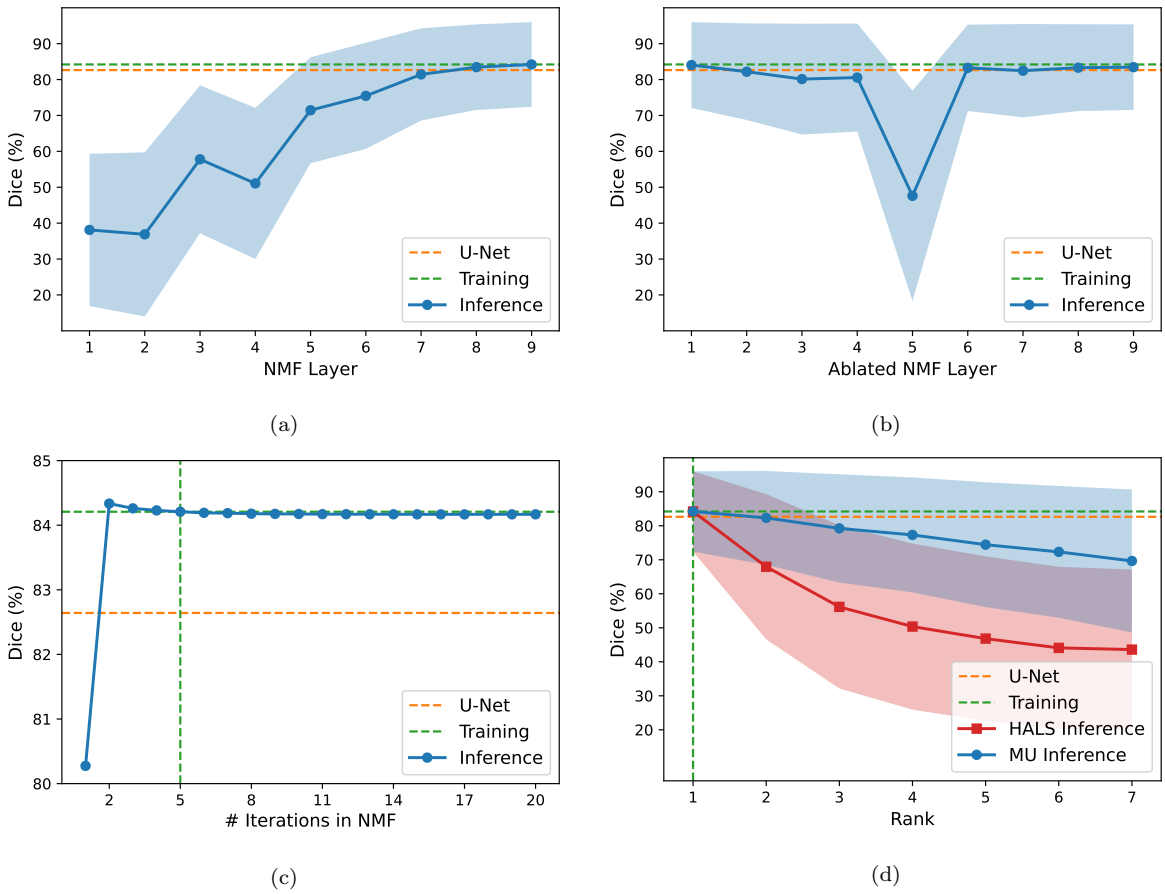


Figure 7 Inference-phase ablation experiments with Swin Factorizer pre-trained on BraTS. Panel (a) shows the average Dice score when the first NMF layers are only kept, while Panel (b) shows the Dice score plotted versus the NMF layer removed. Panel (c) shows the plot of Dice against the number of outer iterations in HALS. Panel (d) shows the impact of rank on the performance in HALS and MU. The dashed orange lines indicate the U-Net Dice score, while the dashed green lines correspond to the pre-trained Swin Factorizer model.

NMF Layer. We investigated the impact of short-circuiting some NMF layers of the pre-trained Swin Factorizer model in the inference phase. In Figure 7a, the average Dice score on BraTS is shown when we kept the first NMF layers and removed (or short-circuit) the rest. As expected, the more layers we kept, the higher Dice score was obtained. We observed that the most of performance is achieved via the encoder, which includes the first five blocks. Figure 7b shows the results of investigating the impact of each individual layer, where we kept all the layers expect one at a time. Interestingly, we noticed that the NMF layer of the bride block (layer 5) makes the greatest contribution to the performance. In fact, if an NMF layer except that of the bridge is ablated, the average Dice still stays above 80%. Particularly, if an NMF layer in the decoder (layers 6 to 9) is ablated, the performance is still better than that of U-Net. Note that removing a NMF layer, especially those at higher-resolution stages of the network, can significantly reduce the computational complexity and speed up the inference time.

NMF Solver Iterations. We investigate the effect of changing the number of outer iterations (T) in HALS. Recall that all the Factorizer models were originally trained using HALS with $T = 5$. Figure 7c shows the results of experimenting with $T \in \{1, \ldots, 20\}$ in the inference phase for the Swin Factorizer model pre-trained with $T = 5$. We noticed

that for $T > 1$, the performance is very close to that of the original model ($T = 5$). Interestingly, $T \in \{2, 3, 4\}$ yielded even higher average Dice scores compared to the original model although the improvement was not statistically significant. We attribute this to the possible regularization effect of reducing T. Note that we proportionally decreased the computational cost of NMF layers by reducing T while preserving the accuracy. This brings a great advantage to Factorizers over CNNs and Transformers, when it comes to model speed-up (discussed further in Section 5).

Rank. We investigated the impact of changing the rank of the pre-trained Swin Factorizer model in the inference phase. Recall that Swin Factorizer were trained with $R = 1$, in which both HALS and MU lead to the same update rule. Therefore, for $R > 1$ in inference, we experimented with both HALS and MU in order to make a comparison. In Figure 7d, the average Dice score on BraTS is plotted as a function of rank. As observed, the more the rank deviated from $R = 1$ (the one used in training), the more significantly the Dice score dropped. In comparison to HALS, MU demonstrated less dramatic reduction in the performance as we increased the rank. This can be due to the fact that within the same number of outer iterations, HALS typically makes a larger decrease in the NMF objective (given in equation (10)), causing the HALS-based model to deviate further from the original model than that of MU.

5. discussion

-Similar to attention, Wrapped NMF is an adaptive filter, which is in contrast to convolution, where the kernel is fixed and independent from the input and does not change after training.

-large window size, $8 \times 8 \times 8$ vs $3 \times 3 \times 3$

-Cheaper from attention (which is computationally intractable for 3D images) and even Performer, as an efficient approximation of attention.

-once a Factorizer model is trained, we have also lightweight yet accurate versions of that for free without needing to re-train any model from scratch. All we need is to reduce T or ablate some costly NMF layers. This is as opposed to CNNs and Tranformers.

-inference-time regularization, generating different models

- possible vanishing gradient, solution: one- or two- step gradient

6. Future Work

Matrix factorization are very flexible versatile

-Automated selection of rank, for example, using sequential/greedy NMF, nonnegative matrix underapproximation. this can benefit especially local factorizers

-Exploit both Local and Global context by customizing NMF.

-tensor factorization

7. Conclusion

Acknowledgements. The research leading to these results has received funding from EU H2020 MSCA-ITN-2018: INtegrating Magnetic Resonance SPectroscopy and Multimodal Imaging for Research and Education in MEDicine (INSPiRE-MED), funded by the European Commission under Grant Agreement #813120. This research also received funding from the Flemish Government (AI Research Program). Sabine Van Huffel and

Pooya Ashtari are affiliated to Leuven.AI - KU Leuven institute for AI, B-3000, Leuven, Belgium.

Appendix

A. Data Augmentation Pipeline

B. Matricize Implementation Details

C. Additional Ablation Studies

References

[1] W. K. Härdle, M. Müller, S. Sperlich, and A. Werwatz, *Nonparametric and semi-parametric models.* Springer Science & Business Media, 2004.

[2] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in *Artificial intelligence and statistics.* PMLR, 2015, pp. 562–570.

[3] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," *arXiv preprint arXiv:2103.14030*, 2021.

[4] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, no. 6755, pp. 788–791, 1999.

[5] L. Grippo and M. Sciandrone, "On the convergence of the block nonlinear gauss–seidel method under convex constraints," *Operations research letters*, vol. 26, no. 3, pp. 127–136, 2000.

[6] N. Gillis, *Nonnegative Matrix Factorization.* SIAM, 2020.

[7] A. Cichocki and A.-H. Phan, "Fast local algorithms for large scale nonnegative matrix and tensor factorizations," *IEICE transactions on fundamentals of electronics, communications and computer sciences*, vol. 92, no. 3, pp. 708–721, 2009.

[8] Z. Geng, M.-H. Guo, H. Chen, X. Li, K. Wei, and Z. Lin, "Is attention better than matrix decomposition?" *arXiv preprint arXiv:2109.04553*, 2021.

[9] B. H. Menze, A. Jakab, S. Bauer, J. Kalpathy-Cramer, K. Farahani, J. Kirby, Y. Burren, N. Porz, J. Slotboom, R. Wiest *et al.*, "The multimodal brain tumor image segmentation benchmark (brats)," *IEEE transactions on medical imaging*, vol. 34, no. 10, pp. 1993–2024, 2014.

[10] S. Bakas, H. Akbari, A. Sotiras, M. Bilello, M. Rozycki, J. S. Kirby, J. B. Freymann, K. Farahani, and C. Davatzikos, "Advancing the cancer genome atlas glioma mri collections with expert segmentation labels and radiomic features," *Scientific data*, vol. 4, p. 170117, 2017.

Algorithm 3: Pytorch Pseudocode of matricize modules.

```python
import torch
from torch import nn
from einops.layers.torch import Rearrange


class LocalMatricize(nn.Module):
    def __init__(self, size, head_dim=8, patch_size=(8, 8, 8)):
        super().__init__()
        C, H, W, D = size
        c2, (h2, w2, d2) = head_dim, patch_size
        c1, h1, w1, d1 = C // c2, H // h2, W // w2, D // d2
        dims = dict(c1=c1, h1=h1, w1=w1, d1=d1, c2=c2, h2=h2, w2=w2, d2=d2)

        eq = "b (c1 c2) (h1 h2) (w1 w2) (d1 d2) -> (b c1 h1 w1 d1) c2 (h2 w2 d2)"
        self.unfold = Rearrange(eq, **dims)

        eq = "(b c1 h1 w1 d1) c2 (h2 w2 d2) -> b (c1 c2) (h1 h2) (w1 w2) (d1 d2)"
        self.fold = Rearrange(eq, **dims)

    def forward(self, x):
        # x: (B, C, H, W, D)
        return self.unfold(x)

    def inverse_forward(self, x):
        # x: (B, C, L)
        return self.fold(x)


class SwinMatricize(nn.Module):
    def __init__(
        self, size, head_dim=8, patch_size=(8, 8, 8), shifts=(4, 4, 4)
    ):
        super().__init__()
        self.local_matricize = LocalMatricize(size, head_dim, patch_size)
        self.shifts = shifts
        self.shifts_inv = tuple(-s for s in patch_size)

    def forward(self, x):
        # x: (B, C, H, W, D)
        out1 = self.local_matricize(x)
        out2 = torch.roll(x, self.shifts, (2, 3, 4))
        out2 = self.local_matricize(out2)
        return torch.cat((out1, out2))

    def inverse_forward(self, x):
        # x: (B, C, L)
        B = x.shape[0]
        out1 = self.local_matricize.inverse_forward(x[: (B // 2)])
        out2 = self.local_matricize.inverse_forward(x[(B // 2) :])
        out2 = torch.roll(out2, self.shifts_inv, (2, 3, 4))
        return 0.5 * (out1 + out2)
```

[11] S. Bakas, M. Reyes, A. Jakab, S. Bauer, M. Rempfler, A. Crimi, R. T. Shinohara, C. Berger, S. M. Ha, M. Rozycki *et al.*, "Identifying the best machine learning al-

gorithms for brain tumor segmentation, progression assessment, and overall survival prediction in the brats challenge," *arXiv preprint arXiv:1811.02629*, 2018.

[12] M. Antonelli, A. Reinke, S. Bakas, K. Farahani, B. A. Landman, G. Litjens, B. Menze, O. Ronneberger, R. M. Summers, B. van Ginneken *et al.*, "The medical segmentation decathlon," *arXiv preprint arXiv:2106.05735*, 2021.

[13] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, pp. 8026–8037, 2019.

[14] N. Ma, W. Li, R. Brown, Y. Wang, Behrooz, B. Gorman, H. Johnson, I. Yang, E. Kerfoot, charliebudd, Y. Li, M. Adil, Y.-T. Hsieh, A. Aggarwal, C. Trentz, adam aji, masadcv, M. Graham, B. Murray, G. Daroach, P.-D. Tudosiu, myron, M. McCormick, Ambros, Balamurali, C. Baker, J. Sellner, L. Fidon, and cgrain, "Project-monai/monai: 0.5.3," Jun. 2021. [Online]. Available: https://doi.org/10.5281/zenodo.4891800

[15] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *2016 fourth international conference on 3D vision (3DV)*. IEEE, 2016, pp. 565–571.

[16] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3d u-net: learning dense volumetric segmentation from sparse annotation," in *International conference on medical image computing and computer-assisted intervention*. Springer, 2016, pp. 424–432.

[17] F. Isensee, P. Kickingereder, W. Wick, M. Bendszus, and K. H. Maier-Hein, "No new-net," in *International MICCAI Brainlesion Workshop*. Springer, 2018, pp. 234–244.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[19] A. Myronenko, "3d mri brain tumor segmentation using autoencoder regularization," in *International MICCAI Brainlesion Workshop*. Springer, 2018, pp. 311–320.

[20] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, "Rethinking attention with performers," *arXiv preprint arXiv:2009.14794*, 2020.